OPENER: Open Partitioning Embedding for Named Entity Recognition

Anonymous Author(s)
Paper submitted for double-blind review
LyRIDS Symposium 2026

Abstract

Open-world named entity recognition (NER) must assign entity types that were not fixed at training time. Recent systems either train a dedicated encoder or prompt billion-parameter language models, and they are compared almost exclusively on accuracy, ignoring their compute and energy cost. We present **OPENER**, a highly accurate open-world NER pipeline that redefines the optimal trade-off between best-in-benchmark accuracy, low latency, and energy frugality, assembled from off-the-shelf components: a frozen mention detector, a Matryoshka-truncatable sentence embedder sharpened by a light contrastive fine-tuning with error-driven hard-negative mining, and a swappable typing head, so that no encoder is trained from scratch. This still-uncommon design reuses pretrained parts rather than the prevailing trained encoders or prompted large language models. The typing head comes in two forms. **OPENER-ZS** is *zero-shot*: it types each mention against label-name prototypes and needs no target labels, for the truly open-world setting where entity types are not known in advance. **OPENER-Sup** instead fits a lightweight supervised probe when a domain's entity classes are fixed and labelled, trading openness for accuracy. We benchmark OPENER against GLiNER, GNER, OWNER and a 4-bit Qwen2.5-1.5B large language model on 13 datasets along three axes at once: **quality** (measured by AMI, for Adjusted Mutual Information), **inference latency** (median per-sentence p50), and **energy consumption** (watt-hours and grams of CO_2eq per run). Throughout, we separate entity typing on gold mentions from the full end-to-end setting, and the energy-aware comparison exposes order-of-magnitude efficiency gaps that a quality-only evaluation would hide. On our 13-dataset benchmark, both forms lead their category. **OPENER-ZS** is the most accurate *zero-shot* system end-to-end (39.4 AMI, ahead of GLiNER-L's 38.9 and OWNER's 34.3) while staying frugal, at 1.7 Wh and $\sim$180 ms per sentence—an order of magnitude below OWNER (14.3 Wh, 616 ms) and the 4-bit Qwen LLM (11.4 Wh, 5 s). Given a domain's labels, **OPENER-Sup** is the most accurate system overall (40.2 end-to-end). On gold mentions, with detection removed, typing alone reaches 62.5 AMI versus OWNER's 43.0: the wide gap to the end-to-end scores pins the open-world bottleneck on mention *detection* rather than typing—the clearest axis for future gains. We release the code, the per-experiment configurations and the full benchmark.

Index Terms

open-world named entity recognition, entity typing, contrastive representation learning, Matryoshka embeddings, frugal artificial intelligence, energy-efficient machine learning, adjusted mutual information, open info extraction, Supervised NER, Zero-Shot NER

I. Introduction

Named entity recognition (NER) in production rarely operates over a fixed inventory of types. New entity categories appear as a system meets new domains, products or users, and a closed-set tagger trained on a frozen label set cannot name a type it has never seen. The standard remedy — re-annotating data for the new types and retraining the model — is slow and expensive, which is precisely what *open-world* NER seeks to avoid: recognising and typing entities whose categories were not fixed at training time.

Existing open-world approaches buy this flexibility at a cost. Span-based closed-set models reach high accuracy but remain bound to their training inventory [1]. OWNER [2] removes that bound by training a dedicated entity encoder with a triplet objective and clustering the resulting mentions into dynamically named types, but it still trains an encoder from scratch and pays a heavy per-sentence inference cost. A complementary line prompts or instruction-tunes billion-parameter language models [3], [4], which are accurate but slow, brittle, and often too large for commodity hardware. Across all of these, evaluation is reported almost exclusively on accuracy: the compute, latency and energy that separate a 137 M-parameter pipeline from a 7 B-parameter model are left out of the comparison, even though they decide what can actually be deployed.

We present **OPENER**, an open-world NER pipeline built for all three axes at once: on our 13-dataset benchmark it is the most accurate of the compared systems while remaining fast and energy-frugal. Unusually, it trains no encoder from scratch—it is assembled entirely from off-the-shelf, pretrained components, a design still uncommon in open-world NER. Concretely, a frozen detector proposes spans, a pretrained Matryoshka sentence embedder — sharpened by a light contrastive fine-tuning and an error-driven hard-negative mining pass — maps each span to a vector, and a typing head assigns a type. The pipeline is modular: detector, embedder and typing head are independently swappable—we vary each in the ablations, comparing GLiNER detectors of three sizes against closed-set NER detectors—and the embedder can be truncated to trade quality for compute. The typing head exposes two operating points. **OPENER-ZS** uses no target labels at typing time: it types each mention against label-name prototypes, refined transductively and *fused* with the detector's own zero-shot predictions—a signal that prototype matching otherwise discards—so it handles the truly open-world setting where entity types are not known in advance. It is the

operating point we put forward: the best end-to-end AMI of all compared zero-shot systems, at an order-of-magnitude-lower energy than the trained-encoder and generative baselines. **OPENER-Sup** instead fits a supervised linear probe when a domain's entity classes are fixed and labelled, trading openness for accuracy: it is the most accurate configuration in our benchmark—the best end-to-end AMI of all compared systems and the best typing on gold mentions—but needs target labels.

Our contributions are as follows.

- **OPENER**, an open-world NER pipeline that replaces task-specific encoder training with *transfer learning*: it reuses off-the-shelf pretrained parts (a frozen detector and a Matryoshka embedder) and only lightly sharpens the embedding, testing whether a general-purpose pretrained representation already transfers well enough to type open-world entities; the typing head is swappable (supervised or zero-shot).
- An *error-driven hard-negative mining* step that fine-tunes the embedding on the type pairs it most often confuses, lifting typing-on-gold AMI by 8.5 points over plain contrastive fine-tuning.
- **OPENER-ZS**, a label-free typing head combining label-name prototypes, a transductive refinement, and a fusion with the detector's zero-shot predictions, which is the best zero-shot system end-to-end on our benchmark, ahead of GLiNER-L, GNER and OWNER under an identical protocol.
- An *energy-aware* benchmark of open-world NER on 13 datasets that reports quality, latency and energy jointly, separates typing-on-gold from end-to-end, and shows that the dominant open-world bottleneck is mention detection rather than typing—and that an *open-world* detector is essential, since closed-set NER detectors (BERT/RoBERTa) collapse on specialized domains—while exposing efficiency gaps that an accuracy-only evaluation would hide. The comparison also includes a small instruction-tuned LLM, Qwen2.5-1.5B-Instruct in 4-bit—the largest generative model that fits our commodity GPU—which turns out to be both the least accurate and the most expensive of all systems.

The rest of the paper is organised as follows. section II reviews related work, section III details the OPENER pipeline, section IV presents the benchmark and ablations, and section V concludes.

II. Related Work

OPENER sits at the intersection of three lines of research: open-world and zero-shot named entity recognition (NER), pretrained entity representations refined by contrastive learning, and energy-aware machine learning. We review each in turn and state how OPENER differs.

A. Closed-set and Span-based NER

Traditional NER is cast as supervised sequence labeling over a fixed inventory of entity types, typically with a pretrained encoder followed by a token- or span-level classifier. Early neural taggers replaced hand-crafted features with recurrent encoders over a CRF layer: Lample et al. [5] paired character- and word-level representations in a BiLSTM-CRF, and Ma and Hovy [6] added a character-level CNN to obtain a fully end-to-end sequence labeler. The pretrained-encoder paradigm subsequently raised in-domain accuracy further, fine-tuning a bidirectional transformer such as BERT [7] with a lightweight classification head. Beyond flat tagging, span-based formulations enumerate candidate spans and classify them directly, which simplifies the handling of nested and discontinuous entities [1]: Yu et al. [8] score every span with a biaffine layer borrowed from dependency parsing, while Li et al. [9] recast typing as machine reading comprehension, querying the sentence with a natural-language description of each type—a use of label semantics our zero-shot head also exploits. These models reach high accuracy on in-domain benchmarks such as CoNLL-2003 [10], as surveyed comprehensively by Li et al. [11], but their label set is frozen at training time. Adding a new entity type requires re-annotating data and retraining the model, which is the central limitation that open-world NER seeks to remove.

B. Open-world and Zero-shot NER

Open-world NER aims to recognise entity types that were not fixed at training time. GLiNER [12] treats the candidate type names as queries to a bidirectional encoder, a DeBERTaV3 backbone [13], and scores every span against them in a single forward pass, offering a fast alternative to autoregressive tagging. GNER [14] instead casts NER as text generation with a sequence-to-sequence model [15] and shows that explicitly modeling negative, non-entity instances sharpens type boundaries. Closest to our work, Genest et al. [2] propose OWNER, an unsupervised open-world pipeline that decouples mention detection from entity typing: it embeds detected mentions with an encoder refined by contrastive learning and then clusters them into dynamically named types. Zero-shot typing has also been driven by the semantics of the type names themselves: Aly et al. [16] were the first to leverage natural-language type *descriptions* so that classes unseen at training time can still be recognised, a premise our label-name prototypes share. Progress in the low-resource regime is in turn anchored by dedicated benchmarks, notably Few-NERD [17], a large human-annotated dataset organised as a two-level hierarchy of 8 coarse and 66 fine-grained types. Other approaches prompt a general-purpose large language model to perform extraction directly, as in ChatIE [18], or focus on recovering entities that detectors tend to miss [19]. OPENER adopts the decoupled detect-then-type view of OWNER, but replaces the trained encoder and the unsupervised clustering with a pretrained, Matryoshka-truncatable embedding space and a balanced linear classifier, so that no encoder is trained from scratch.

C. LLM-based and Few-shot NER

A complementary line distils or instruction-tunes large language models for open NER. UniversalNER [3] distils a chat model into a smaller LLaMA-based student [20] over tens of thousands of entity types, while GoLLIE [4] conditions a code language model on annotation guidelines to follow unseen schemas. Few-shot methods reduce supervision further through prompt optimization [21], knowledge-graph-guided contrastive learning [22], or token-level contrastive objectives such as CONTaiNER [23]. These methods are accurate but rest on models with billions of parameters, whose inference cost, latency and energy use are substantial, and which may not even fit on commodity hardware. At the small end of this family, however, a quantized instruction-tuned model does fit a commodity GPU: as a concrete, runnable reference point we include Qwen2.5-1.5B-Instruct [24] in 4-bit, prompted to return entities as JSON, and report where it lands relative to the dedicated open-world systems (subsection IV-F). This cost is precisely what motivates the lightweight design and the energy-aware evaluation of OPENER.

D. Entity Representations and Contrastive Learning

OPENER relies on a pretrained sentence embedder rather than a task-specific encoder. Nomic Embed [25] provides an open, long-context model trained with the Matryoshka objective [26], whose representations can be truncated to lower dimensions without retraining, trading quality for compute. Such embedders build on the Sentence-BERT line of work [27], and we sharpen the entity-type geometry with a triplet margin objective in the spirit of FaceNet [28]. The choice of base encoder is made empirically: we compare several strong open sentence encoders—E5 [29], BGE [30], MPNet [31] and mxbai [32]—on entity typing under a common protocol (Table VIII); Nomic v1.5 [25] edges them out while additionally offering native Matryoshka truncation, so we adopt it as the base and fine-tune it. Embedding labels and entities in a shared space [33], modeling entity types with Gaussian components [34], generating synthetic entity-bearing text for augmentation [35], and using mixtures to augment training data [36] are recurring ideas in low-resource NER, all grounded in classical Gaussian mixture models fitted by expectation-maximization [37], [38]. Unlike methods that train a dedicated encoder, OPENER takes transfer learning in its purest form: it keeps a general-purpose pretrained space largely frozen and only applies a light contrastive fine-tuning, testing whether such a representation is already discriminative enough for entity typing.

E. Efficient and Green AI

A growing body of work argues that efficiency should be reported alongside accuracy. Strubell et al. [39] quantify the financial and environmental cost of training NLP models, Schwartz et al. [40] advocate efficiency as a first-class evaluation criterion, and Patterson et al. [41] analyse the carbon footprint of machine-learning training and how to reduce it. Henderson et al. [42] argue for the systematic reporting of energy and carbon and provide a framework to track them per run, while Wu et al. [43] take a whole-lifecycle, system-level view of AI's environmental footprint across data, algorithms and hardware. Most relevant to deployment, Luccioni et al. [44] measure *inference*-time cost and find general-purpose generative models to be orders of magnitude more expensive per query than task-specific ones—precisely the gap a lightweight pipeline like ours seeks to close. Tools such as CodeCarbon [45] make per-run energy and carbon estimates practical. Open-world NER systems, however, are still compared almost exclusively on quality. OPENER instead reports a three-axis comparison of quality, measured by Adjusted Mutual Information [46], inference speed, and energy, placing the frugality of each method on an equal footing with its accuracy.

In summary, existing open-world NER methods either train a dedicated encoder [2] or query costly large language models [3], [4], and they are compared on accuracy alone. OPENER fills both gaps: it reuses pretrained components with only a light contrastive step and a balanced linear classifier, and it is evaluated jointly on quality, speed and energy.

III. Method

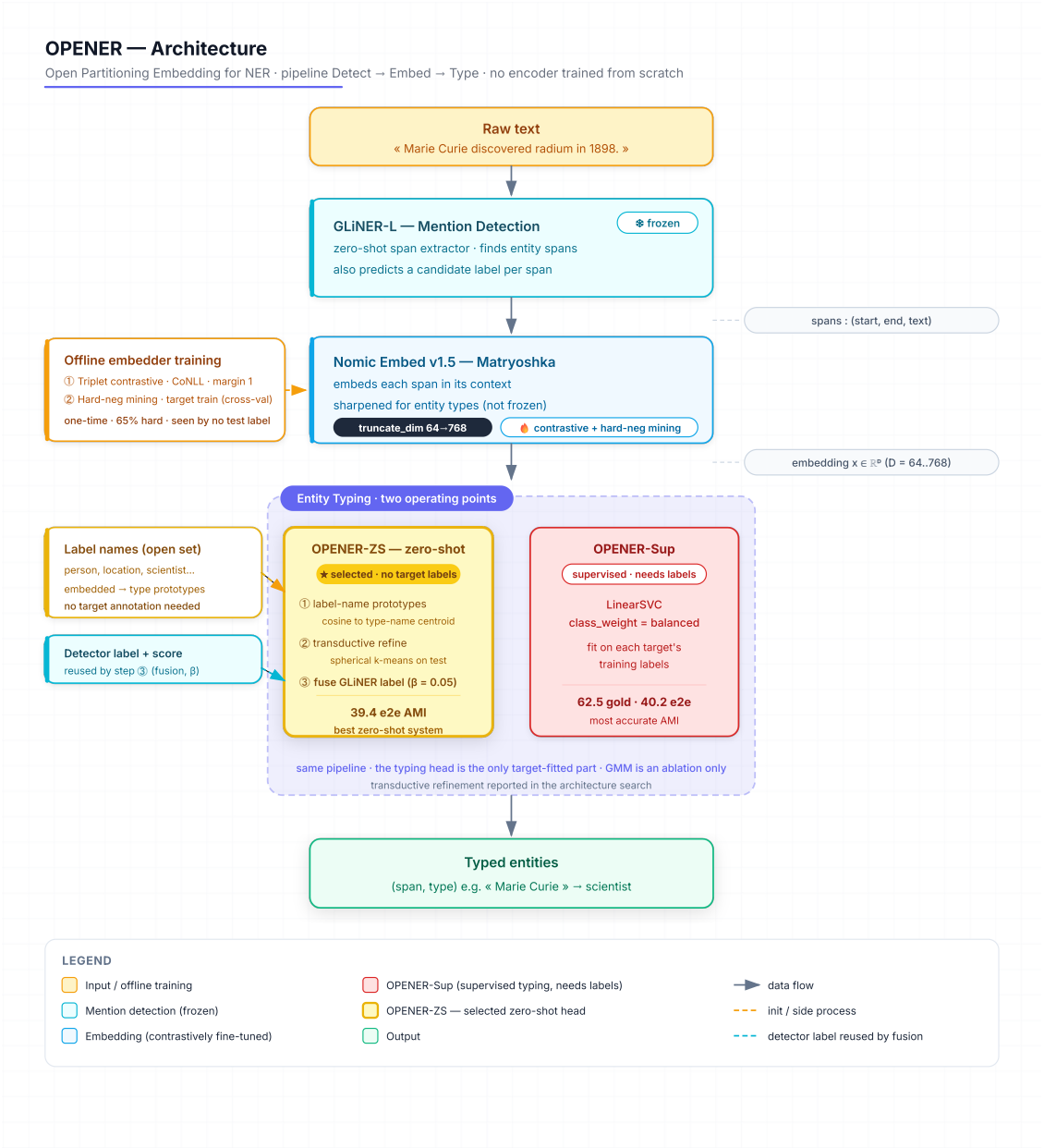

Fig. 1. The OPENER pipeline (Detect → Embed → Type): a frozen GLiNER detector proposes spans, a Matryoshka embedder (contrastively fine-tuned) maps each span in context to a vector, and a typing head assigns a type. The typing head—the only target-fitted component—has two operating points: **OPENER-Sup** (*supervised*, the upper bound) and the *zero-shot* **OPENER-ZS** (★, selected). All upstream components are off-the-shelf.

A. Problem Formulation

Given a sentence, an open-world NER system must return a set of typed mentions whose type inventory is *not* fixed at training time. Formally, a sentence s yields candidate mentions $\mathcal{M}(s) = \{m_i\}$; an embedder f_θ maps each mention *in context* to a vector and a typing head assigns it a type,

$$e_i = f_\theta(s, m_i) \in \mathbb{R}^D, \qquad \hat{y}_i \in \mathcal{T}, \tag{1}$$

where the inventory $\mathcal{T}$ is unknown at training time. We adopt the evaluation of Genest et al. [2]. Predicted and gold mentions are aligned by exact character offsets; an unmatched gold mention (false negative) and an unmatched predicted mention (false positive) each receive a distinct sentinel type, so that detection errors are scored rather than ignored. Type-assignment quality

is then measured by the Adjusted Mutual Information (AMI) [46] between the gold type partition U and the predicted partition V over the aligned mentions,

$$\mathrm{AMI}(U,V) = \frac{\mathrm{MI}(U,V) - \mathbb{E}[\mathrm{MI}(U,V)]}{\max\{H(U),H(V)\} - \mathbb{E}[\mathrm{MI}(U,V)]}, \tag{2}$$

where MI is mutual information and H entropy; AMI does not require the two label vocabularies to coincide. We report two protocols that are never mixed in the same comparison: *typing-on-gold*, where the typing component is applied to gold spans and detection is bypassed, and *end-to-end*, where mentions come from the detector and the sentinels above are active.

B. Architecture Overview

OPENER is a three-stage pipeline, Detect $\rightarrow$ Embed $\rightarrow$ Type (Figure 1), assembled from off-the-shelf components so that no encoder is trained from scratch. Three principles guide the design. (i) A pretrained detector and a pretrained sentence embedder are reused as-is. (ii) A light contrastive fine-tuning, refined by error-driven hard-negative mining, turns the general-purpose embedding into one that is discriminative for entity *types*. (iii) The embedder is Matryoshka-truncatable, exposing a compute/quality knob. Only the typing head is fitted on target-side material, and it comes in two forms: **OPENER-Sup**, a supervised linear probe that is the strongest configuration but requires target labels, and a zero-shot form, **OPENER-ZS**, that uses no target labels at typing time.

C. Mention Detection

Mention detection is delegated to GLiNER [12], a frozen, off-the-shelf detector that scores every span against the candidate type names in a single forward pass. Decoupling detection from typing lets each component be chosen and measured independently, and it lets the typing head operate on any span proposer. We use GLiNER-L by default; the small and medium variants trace a size/efficiency trade-off (Table VIII). As shown in subsection IV-F, the detector is the shared end-to-end bottleneck: its recall on specialised domains, not the typing head, caps end-to-end quality.

D. Entity Embedding

Detected spans are embedded with Nomic Embed v1.5 [25], a sentence embedder trained with the Matryoshka objective [26], whose representation can be truncated to a lower dimension and renormalised without retraining. Each mention is embedded *in context* rather than in isolation: the span is presented together with its surrounding sentence, so that polysemous surface forms (e.g. a company versus a fruit) are disambiguated. This yields a vector $e_i \in \mathbb{R}^D$ with D truncatable in $\{768, 512, 256, 128, 64\}$; we keep the full $D{=}768$ by default. Truncation is a post-encoding slice of the first d coordinates followed by renormalisation,

$$e_i^{(d)} = e_{i,1:d} \,/\, \|e_{i,1:d}\|_2, \tag{3}$$

so it reduces the stored embedding and the typing-head fit cost but leaves the forward pass unchanged (Table VIII).

E. Contrastive Fine-tuning with Hard-Negative Mining

The frozen embedding is only weakly discriminative for entity types (25.8 AMI, Table VIII), so we sharpen its geometry in two stages while keeping the backbone otherwise intact.

a) Stage 1 – contrastive: We fine-tune the embedder with a Triplet Margin Loss [28] (using sentence-transformers [27]), with margin 1, on the CoNLL-2003 [10] training spans. A triplet (a, p, n) takes an anchor span a, a positive p of the *same* type, and a negative n of a *different* type, with at most two triplets per anchor. Writing α for the margin, each triplet contributes

$$\mathcal{L}(a,p,n) = \max\big(0,\ \|e_a - e_p\|_2^2 - \|e_a - e_n\|_2^2 + \alpha\big), \quad \alpha = 1, \tag{4}$$

which pulls same-type spans together and pushes different-type spans apart. Training runs for 3 epochs with batch size 16, learning rate 2×10^{-5} and 100 warmup steps. This single step lifts typing-on-gold AMI from 25.8 to 54.0.

b) Stage 2 – error-driven hard-negative mining: We then mine the model's own confusions. On each target training split we run a cross-validated typing pass (so no test label is ever seen) and collect the type pairs that are most often confused, such as *writer*/*person*, *album*/*band* and *country*/*location*. For each such error we form a *hard* triplet whose anchor is the mistyped span (true type L), whose positive is another span of L, and whose negative is a span of the confused type C. Hard triplets are mixed with correctly-typed (easy) triplets for diversity, targeting a 65% hard fraction, and the Stage-1 embedder is fine-tuned further with the same optimiser settings. The selected run mines 8000 triplets over 3 epochs and reaches 62.5 AMI (+8.5 over Stage 1, better on 12 of 13 sets). This embedder is shared unchanged by every typing head below.

F. Entity Typing

a) Supervised probe (OPENER-Sup): The default typing head is a balanced linear classifier (LinearSVC with the LIBLINEAR solver [47]) fitted one-vs-rest on the target's training labels. For each type c it minimises a squared-hinge objective with balanced class weights,

$$\min_{w_c, b_c} \ \tfrac{1}{2}\|w_c\|_2^2 + C \sum_i \omega_{y_i} \ \max\bigl(0,\, 1 - z_{ic}(w_c^\top e_i + b_c)\bigr)^2, \quad \omega_c = \frac{n}{|\mathcal{T}|\, n_c}, \tag{5}$$

where $z_{ic}{=}{+}1$ if $y_i{=}c$ and -1 otherwise, n_c is the count of type c, and the weights ω_c counter the benchmark's heavy type imbalance; a mention is typed as $\hat{y}_i = \arg\max_c (w_c^\top e_i + b_c)$. This is the strongest configuration but requires target-side labels.

b) Zero-shot typing (OPENER-ZS): To remove target supervision at typing time, each type t is represented by a prototype p_t: the ℓ_2-normalised centroid of the embeddings of its name (its anchor words) in the contrastive space,

$$p_t = \bar{e}_t \,/\, \|\bar{e}_t\|_2, \qquad \bar{e}_t = \sum_{w \,\in\, \mathrm{name}(t)} f_\theta(w), \tag{6}$$

and a mention is assigned the type of the nearest prototype by cosine similarity. Two label-free refinements improve this head without using any target label. First, a *transductive* refinement runs one pass of spherical k-means over the unlabelled test mentions, seeded by the prototypes. Second, a *detector fusion* reuses a signal the prototype head otherwise discards: GLiNER already predicts a zero-shot type for each span with confidence g_i, which we add to the similarity score before taking the arg-max,

$$s_t(i) = \cos(e_i, p_t) + \beta\, g_i \mathbb{1}[t = \ell_{\text{GLiNER}}(i)], \qquad \hat{y}_i = \arg\max_t s_t(i), \tag{7}$$

with a small $\beta{=}0.05$. The fused head is the selected zero-shot operating point: it reaches 39.4 AMI end-to-end, ahead of every compared zero-shot system, while using no target labels (subsection IV-F). A per-class Gaussian mixture [37] is reported only as a typing ablation (Table VIII).

G. Evaluation Protocol

We evaluate on the 13 datasets of subsection IV-A, each test split capped at 1000 sentences. Quality is AMI [46] (with macro-F_1 and accuracy), latency is the median (p50) per-sentence time over GPU-synchronised, warmed-up passes, and energy is measured with CodeCarbon [45] under the French grid carbon intensity; the full protocol and instrumentation are detailed in subsection IV-D and subsection IV-E. The code and the per-experiment configurations are released for reproducibility.

IV. Experiments

A. Datasets

TABLE I
THE 13 EVALUATION SETS AND THEIR GOLD ENTITY TYPES. "TEST" IS THE NUMBER OF TEST SENTENCES USED (CAPPED AT 1000); EACH TYPE IS FOLLOWED BY ITS SHARE (%) OF THE TEST MENTIONS, LISTED BY DECREASING FREQUENCY (TYPES ARE THE DATASET'S ORIGINAL LABELS).

Dataset	Abbr.	Domain	Test	Entity types (share %)
CrossNER-AI	AI	Encyclopedic (AI)	430	task 12, field 11, product 11, metrics 11, misc 10, algorithm 10, researcher 9, organisation 8, conference 5, person 4, programlang 3, country 2, location 2, university 2
CrossNER-Literature	Lit	Encyclopedic (literature)	416	writer 25, book 18, misc 11, literarygenre 9, person 8, award 6, poem 5, organisation 5, country 4, location 4, magazine 3, event 2
CrossNER-Music	Mus	Encyclopedic (music)	465	musicalartist 15, musicgenre 15, band 14, location 10, album 10, award 8, song 7, organisation 6, misc 5, country 5, person 2, event 2, musicalinstrument 1
CrossNER-Politics	Pol	Encyclopedic (politics)	651	politicalparty 23, location 14, organisation 12, politician 12, election 10, country 10, person 8, misc 6, event 5
CrossNER-Science	Sci	Encyclopedic (science)	543	misc 17, scientist 15, astronomicalobject 11, organisation 9, person 7, academicjournal 6, location 6, chemicalcompound 5, protein 5, award 5, university 4, chemicalelement 3, enzyme 3, discipline 2, event 1, country 1, theory 1
WNUT-17	WNUT	Social media	689	person 40, group 15, location 14, creative-work 13, product 12, corporation 6
MIT-Restaurant	Rest	Restaurant search	1000	Location 21, Dish 18, Amenity 16, Hours 15, Restaurant_Name 11, Price 10, Cuisine 5, Rating 4
MIT-Movie	Movie	Movie trivia	1000	TITLE 19, ACTOR 16, YEAR 16, GENRE 16, RATING 9, CHARACTER 9, TRAILER 5, REVIEW 3, RATINGS_AVERAGE 3, PLOT 2, DIRECTOR 2, SONG 1
FabNER	Fab	Manufacturing	1000	MATE 27, CONPRI 21, MANP 15, PRO 11, APPL 6, FEAT 6, PARA 4, MACEQ 3, BIOP 2, CHAR 2, ENAT 2, MANS <1
BioNLP/JNLPBA-2004	Bio	Biomedical	1000	protein 53, cell_type 22, DNA 13, cell_line 10, RNA 1
CoNLL-2003	CoNLL	News	1000	PER 32, ORG 29, LOC 28, MISC 11
GUM	GUM	Multi-genre	1000	abstract 38, person 17, event 12, place 10, object 8, organization 8, time 5, substance 1, animal 1, plant <1
GENTLE	GEN	Multi-genre (challenge)	1000	abstract 55, person 10, event 9, time 8, object 7, place 5, organization 4, animal 1, substance 1, plant <1

We evaluate on **13 evaluation sets** that span news, social media, spoken queries, encyclopedic text, biomedical abstracts and manufacturing literature, following the dataset selection of Genest et al. [2]. CrossNER [48] contributes five domain-specific sets, evaluated separately (artificial intelligence, literature, music, politics, science). The remaining eight are CoNLL-2003 [10], WNUT-17 [49], the MIT-Restaurant and MIT-Movie spoken-query corpora [50], FabNER [51], the BioNLP/JNLPBA-2004 biomedical task [52], and the GUM [53] and GENTLE [54] multi-genre corpora. Label granularity ranges from four coarse types in CoNLL-2003 to twelve fine-grained categories in FabNER, including opaque domain acronyms. Two datasets used by [2], GENIA [55] and i2b2, are license-gated and therefore not re-measured here; BioNLP/JNLPBA-2004 serves as our biomedical proxy. To homogenise evaluation cost across systems, we cap each test split at 1000 sentences (the full split is used when it is smaller).

B. Baselines

TABLE II
BACKBONES AND SIZE OF THE RE-MEASURED BASELINES AND OPENER, IN THE SPIRIT OF GENEST ET AL. [2]. "TYPE" IS THE TRANSFORMER FAMILY; "VS OPENER" IS THE TOTAL PARAMETER COUNT RELATIVE TO OPENER'S END-TO-END PIPELINE (467M = GLINER-L 330M DETECTOR + NOMIC 137M EMBEDDER). † OWNER REPORTS 110M FOR ITS BERT TYPING ENCODER ALONE; WE COUNT DETECTION + TYPING FOR PARITY WITH OPENER.

Model	Backbone	Type	Version	Params	vs OPENER
GLiNER-S	DeBERTaV3	Encoder	deberta-v3-small	50M	0.1×
GLiNER-M	DeBERTaV3	Encoder	deberta-v3-base	200M	0.4×
GLiNER-L	DeBERTaV3	Encoder	deberta-v3-large	330M	0.7×
GNER	T5	Enc.–Dec.	t5-base	220M	0.5×
OWNER†	DeBERTaV3+BERT	Encoder	deberta-v3 + bert-base	294M	0.6×
Qwen2.5-1.5B	Qwen2.5	Decoder	qwen2.5-1.5b-instruct	1.5B	3.2×
OPENER (ours)	DeBERTaV3+Nomic	Encoder	deberta-v3-large + nomic-v1.5	467M	1.0×

We compare OPENER against representative open-world systems, all run end-to-end on the same splits and scored with the same protocol (subsection IV-D); Table II summarises their backbones and sizes.

- **GLiNER** [12]: an open-world span classifier on a DeBERTaV3 encoder [13]; we run all three sizes (S/M/L) to trace a size/efficiency trade-off.
- **GNER** [14]: a generative tagger built on T5 [15]; we run the T5-base checkpoint.
- **OWNER** [2]: our closest baseline, a DeBERTaV3 detector with a BERT typing encoder [7] clustered into dynamically named types; executed in its original environment.
- **Qwen2.5-1.5B-Instruct** [24]: the only generative LLM that fits our 6 GB GPU, run in 4-bit NF4 [56] and prompted for JSON-formatted entities.
- **Large LLM systems**: UniversalNER [3], GoLLIE [4] and ChatIE [18] rely on 7B-parameter backbones [20] that exceed our hardware even under 4-bit quantization; we report the figures published by their authors rather than re-measuring them, and omit them from Table II.

C. Metrics

Our primary metric is the Adjusted Mutual Information (AMI) [46] between predicted and gold type assignments, following [2], complemented by macro-averaged F_1 and accuracy. Because open-world predictions need not share the gold label vocabulary, AMI is computed over the aligned set of mentions, which makes it well suited to comparing dynamically named types. Beyond quality, we report two efficiency axes: inference *speed*, as the median (p50) latency per sentence, and *energy*, as kilowatt-hours and grams of CO_2eq per run.

D. Evaluation Protocol

Predicted and gold mentions are aligned by exact character offsets. Following [2], an unmatched gold mention (a false negative) and an unmatched predicted mention (a false positive) are each assigned a distinct sentinel label before computing AMI, so that detection errors are penalised rather than ignored. We distinguish two protocols that must not be mixed in the same comparison. In the *typing-on-gold* protocol, the entity-typing component is applied to gold mention spans, which measures typing quality in isolation under perfect detection. In the *end-to-end* protocol, mentions are produced by the detector and the sentinel labels above are active, which is the setting comparable to the baselines. All runs use a single fixed random seed (42); we note this as a limitation, since we do not report variance over seeds.

E. Experimental Settings

All experiments run on a single consumer-grade GPU with 6 GB of VRAM (CUDA 12.1, PyTorch 2.5.1 [57]). We use the HuggingFace Transformers library [58] for the baselines, sentence-transformers [27] for the Matryoshka embedder [25], [26], and scikit-learn [59] for the linear classifier, whose solver is LIBLINEAR [47]. Latency is measured with two warmup passes followed by timed passes with GPU synchronisation, reporting p50, p95 and p99 per sentence. Energy is measured with CodeCarbon [45] in offline mode using the French grid carbon intensity; when hardware power counters are unavailable, we fall back to a thermal-design-power estimate and flag the measurement method in the report. All systems are measured on the same machine. Because the laptop-class GPU thermally throttles under sustained load, per-sentence latency and per-run energy carry roughly $\pm 15\%$ run-to-run variance; we therefore read the per-system averages as indicative and do not over-interpret individual per-dataset cells. A controlled-temperature re-measurement of all systems in a single session is left to future work. The quality metrics, in contrast, are fully deterministic: with the fixed seed and the released model, no stage of the OPENER pipeline is stochastic at inference, so its AMI and F_1 reproduce exactly across re-runs (we verified identical CrossNER scores

TABLE III
OPENER CONFIGURATION. THE DETECTOR AND EMBEDDER ARE OFF-THE-SHELF; THE CONTRASTIVE AND HARD-MINING STAGES PRODUCE THE SHARED EMBEDDER; THE TYPING HEAD IS FITTED (SUPERVISED) OR LABEL-FREE (ZERO-SHOT).

Component	Setting
Mention detector	GLiNER-L (frozen) [12]
Entity embedder	Nomic v1.5, D=768, span-in-context [25]
Contrastive loss	Triplet margin 1 [28]
source	CoNLL-2003 train spans [10]
optimiser	3 epochs, batch 16, lr 2×10^{-5}, 100 warmup
Hard-neg. mining	8000 triplets, 65% hard, 3 epochs
source	target train splits, cross-validated
Supervised typing	LinearSVC, balanced weights [47]
Zero-shot typing	label-name prototype + transductive + fusion (β=0.05)
Random seed	42

on an independent re-run), and only latency and energy fluctuate. Table III collects the full OPENER configuration used throughout; all stochastic components use the single fixed seed (42).

F. Main Results

The tables below report the benchmark on all 13 sets, and Figure 2 plots the per-dataset end-to-end AMI of every system. OWNER is run in its native protocol [2]: a single model is trained once on the CoNLL-2003 source (a one-time cost of 88 minutes on our 6 GB GPU) and then applied *zero-shot* to every target set (cells marked §). CoNLL-2003 is therefore in-domain for OWNER (marked †) and is excluded when reading OWNER as a transfer system. One protocol asymmetry must be kept in mind when reading the typing scores: OPENER-Sup fits a lightweight linear classifier on each target's training split (a supervised probe), whereas OWNER, GLiNER and GNER use no target-side labels. The two systems thus trade annotation for inference cost: OPENER needs target labels but stays cheap at test time, while OWNER needs none but pays a heavy per-sentence cost—it re-encodes every mention and clusters at inference time—an order of magnitude above OPENER (Table V, Table VI). To close this asymmetry we also evaluate OPENER-ZS, a zero-shot variant that drops the supervised probe and types each mention against label-name prototypes, using no target labels. Its un-refined nearest-centroid baseline, OPENER-ZS (base), already matches the zero-shot OWNER baseline end-to-end (34.2 vs 34.3 mean AMI) and edges it on gold (44.4 vs 43.0, Table IX), while keeping OPENER's frugal inference profile (137 ms, 1.85 Wh vs 616 ms, 14.3 Wh). Two label-free refinements then build the full OPENER-ZS: a transductive k-means on the prototypes, and *fusing in the detector's own zero-shot label predictions* — a signal the centroid otherwise discards — which together reach 39.4 end-to-end, the best of all compared zero-shot systems, ahead of GLiNER-L (38.9) and within 0.8 AMI of OPENER-Sup, all without any target label. For reference, the only generative LLM that fits our 6 GB GPU — Qwen2.5-1.5B-Instruct [24] in 4-bit, prompted for JSON-formatted entities — trails every dedicated system (26.1 mean AMI on a 150-sentence subsample, Table IV) while being the slowest and most energy-hungry per sentence (5.1 s, 11.4 Wh; Table VII): a small open LLM pays on both axes at once. OPENER-Sup remains the stronger configuration (40.2 end-to-end, 62.5 on gold), so OPENER-ZS is best read as the frugal, annotation-free operating point of the same pipeline. A per-domain analysis of these results, the detection bottleneck and the learned embedding geometry follows in subsection IV-G.

[REMINDER (draft): consider adding Pareto diagrams (AMI vs. latency, AMI vs. energy consumption) to visualise the frugality frontier; decide before submission.]

G. Analysis

a) Where OPENER wins, and why: Two domain patterns stand out across Table IV and Figure 2. First, the detector-fusion gains of OPENER-ZS concentrate on the context-rich encyclopedic CrossNER domains (+16.3 AMI on Music, +11.9 on Politics, +8.6 on Science over the un-fused centroid), where the detector's own zero-shot label is most reliable and lifts OPENER-ZS to detector-level quality (e.g. Music 58.0, matching GLiNER-L). Second, on specialised schemas with opaque, abbreviation-like labels (FabNER, BioNLP), OPENER's contrastive typing maps cryptic surface forms far better than OWNER's prompt-based naming: OPENER-Sup leads end-to-end (30.2 and 36.5 AMI), and on gold mentions the gap widens sharply (59.2/71.2 versus OWNER's 31.9/29.2, Table IX).

b) Detection is the bottleneck, worst on specialised domains: Contrasting typing on gold mentions with the end-to-end scores isolates the cost of detection. For OPENER-Sup, the mean drop from gold to end-to-end is 22.3 AMI, but it is far from uniform: it averages 18.6 on the encyclopedic CrossNER domains against 30.6 on the specialised schemas (BioNLP, MIT-Restaurant, MIT-Movie, FabNER), peaking at 34.7 on BioNLP. The typing head is strong everywhere (gold AMI 50–75);

TABLE IV

OPEN-WORLD NER, END-TO-END. AMI (×100, ↑); ABBREVIATIONS FOLLOW TABLE I; BOLD MARKS THE BEST VALUE PER COLUMN ACROSS ALL SYSTEMS; SYSTEMS ARE GROUPED BY SUPERVISION (*zero-shot* VS THE *supervised* OPENER-SUP). † CONLL-2003 IS IN-DOMAIN FOR OWNER (TRAINING SOURCE) AND OPENER (CONTRASTIVE SOURCE) AND IS NOT BOLDED AS A TRANSFER RESULT. ‡ QWEN2.5-1.5B-INSTRUCT (4-BIT) IS ON A 150-SENTENCE SUBSAMPLE (INDICATIVE, NOT BOLDED). TYPING-ON-GOLD IS REPORTED IN TABLE IX.

Model	AI	Lit	Mus	Pol	Sci	WNUT	Rest	Movie	Fab	Bio	CoNLL	GUM	GEN	Avg
Zero-shot (no target labels)														
GLiNER-S	41.7	49.8	56.4	50.6	53.3	29.7	26.1	35.6	10.7	31.8	18.9	**34.1**	**36.1**	36.5
GLiNER-M	42.0	**50.1**	57.4	50.2	**54.2**	28.1	27.9	36.4	11.3	34.9	24.9	32.8	31.6	37.1
GLiNER-L	42.9	48.9	**58.0**	49.8	52.8	28.9	30.9	37.6	28.1	34.6	29.7	32.7	30.5	38.9
GNER	41.9	46.9	53.5	48.9	49.9	**30.0**	30.0	36.8	17.0	29.1	**43.7**	32.8	31.6	37.9
Qwen2.5-1.5B‡	25.4	30.5	30.5	28.3	34.1	27.8	5.0	27.4	12.9	29.9	15.5	37.2	34.6	26.1
OWNER	**43.9**	44.0	46.6	**50.9**	50.3	22.3	7.6	24.0	18.7	26.5	50.3†	29.1	31.3	34.3
OPENER-ZS (base)	31.6	43.8	41.7	36.7	44.2	23.0	28.4	32.4	28.3	29.1	42.6†	31.5	31.2	34.2
OPENER-ZS	39.4	48.8	**58.0**	48.6	52.8	29.6	31.5	35.5	27.8	35.7	43.1†	31.2	30.4	39.4
Supervised (target labels)														
OPENER-Sup	40.2	49.6	55.3	50.3	51.7	24.9	**33.6**	**37.8**	**30.2**	**36.5**	44.8†	33.7	34.2	**40.2**

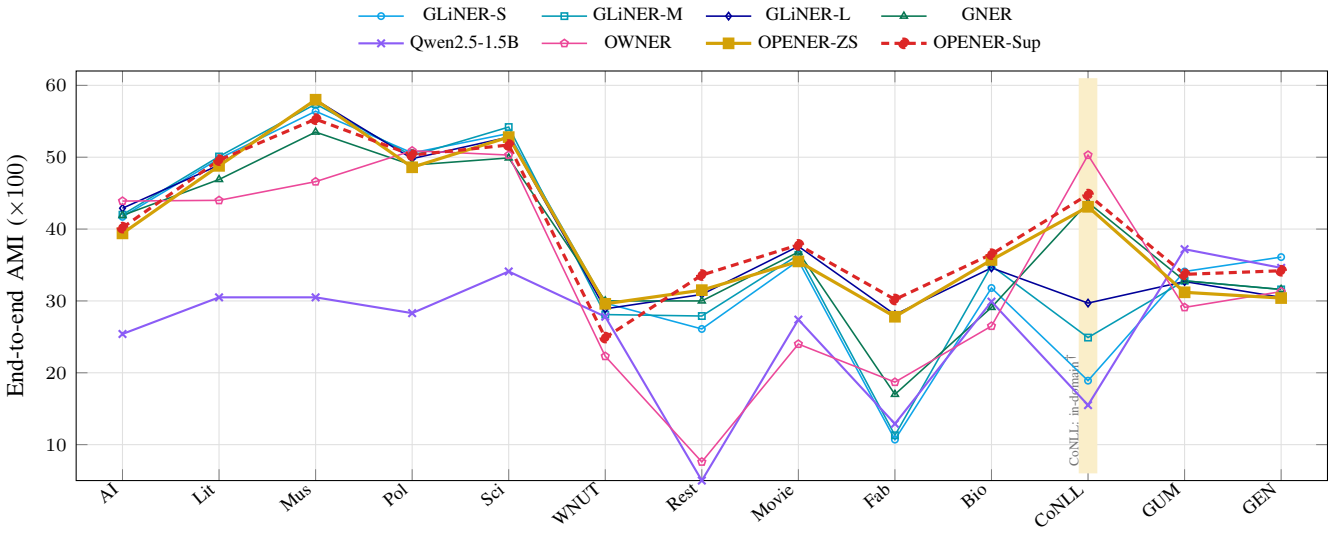

Fig. 2. End-to-end AMI per dataset (Table IV). *Zero-shot* systems are drawn solid; OPENER-Sup (the *supervised* upper bound) is the dashed red line. † CoNLL-2003 (shaded) is in-domain for OWNER and OPENER; ‡ Qwen is evaluated on a 150-sentence subsample per set.

TABLE V

MEDIAN PER-SENTENCE LATENCY (MS, ↓), END-TO-END; ABBREVIATIONS FOLLOW TABLE I, BEST PER COLUMN IN BOLD. § OWNER EXPOSES NO PER-SENTENCE CALL: ITS FIGURE IS AN AMORTISED WALL-CLOCK MEAN (ITS 88-MIN TRAINING IS EXCLUDED); † CONLL IS OWNER'S IN-DOMAIN SOURCE, MEASURED BY A SEPARATE INFERENCE-ONLY RUN; ‡ QWEN2.5-1.5B-INSTRUCT (4-BIT) IS ON A 150-SENTENCE SUBSAMPLE PER SET.

Model	AI	Lit	Mus	Pol	Sci	WNUT	Rest	Movie	Fab	Bio	CoNLL	GUM	GEN	Avg
Zero-shot (no target labels)														
GLiNER-S	**21**	**22**	**21**	**21**	**21**	**20**	**19**	**20**	**20**	**22**	**20**	**21**	**21**	**21**
GLiNER-M	36	36	37	38	39	39	38	40	41	42	44	46	45	40
GLiNER-L	144	149	153	149	155	137	132	145	147	143	131	141	137	143
GNER	3431	5408	7461	6472	7110	2906	1610	3344	4310	5034	2914	3435	1500	4226
Qwen2.5-1.5B‡	4857	6757	9396	7700	8261	3553	2142	5014	2921	4737	2260	4265	3806	5051
OWNER§	500	712	895	920	757	311	281	532	438	558	719†	759	626	616
OPENER-ZS (base)	109	135	156	172	175	136	127	137	133	136	117	127	124	137
OPENER-ZS	218	254	291	277	289	185	173	205	84	98	92	104	97	182
Supervised (target labels)														
OPENER-Sup	222	271	293	159	132	100	92	98	104	108	89	100	94	143

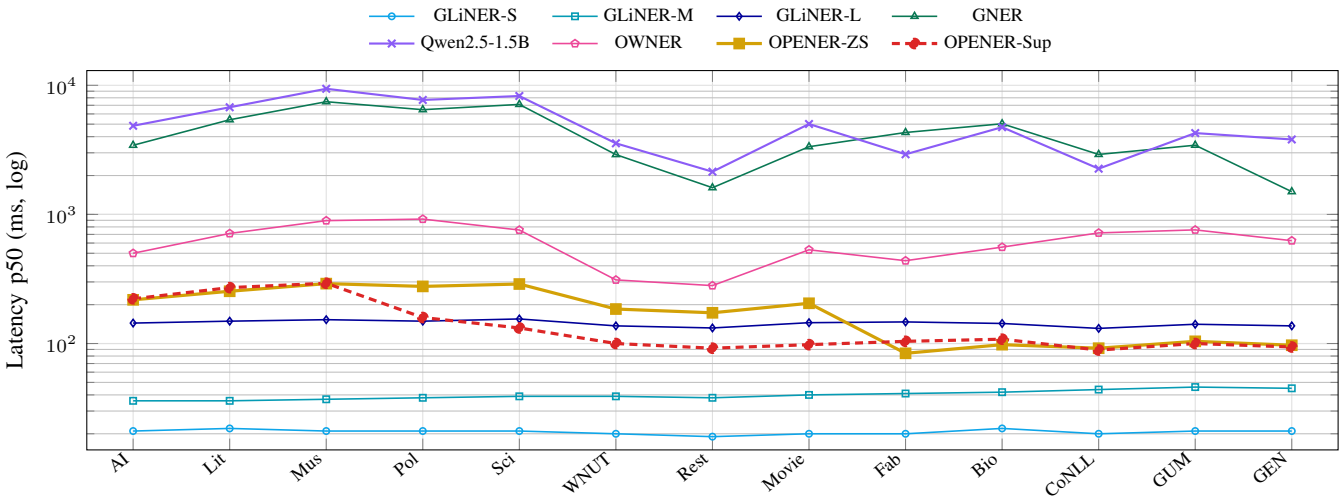

Fig. 3. Per-sentence inference latency (p50, ms, *log* scale) per dataset (Table V). Same systems and styling as Figure 2; OPENER-Sup is the dashed red line. The OPENER variants track GLiNER-L, while GNER and the Qwen LLM are one to two orders of magnitude slower.

TABLE VI

ENERGY CONSUMPTION PER RUN (WH, ↓), END-TO-END. ABBREVIATIONS FOLLOW TABLE I; BEST PER COLUMN IN BOLD. § OWNER IS MEASURED AT INFERENCE ONLY (ITS ONE-TIME 88-MIN CONLL-2003 TRAINING IS AMORTISED, NOT CHARGED PER RUN); † CONLL IS ITS IN-DOMAIN TRAINING SOURCE, MEASURED BY A SEPARATE INFERENCE-ONLY RUN; ‡ QWEN2.5-1.5B-INSTRUCT (4-BIT) IS ON A 150-SENTENCE SUBSAMPLE PER SET.

Model	AI	Lit	Mus	Pol	Sci	WNUT	Rest	Movie	Fab	Bio	CoNLL	GUM	GEN	Avg
Zero-shot (no target labels)														
GLiNER-S	**0.19**	**0.17**	**0.20**	**0.27**	**0.22**	**0.27**	**0.35**	**0.39**	**0.40**	**0.42**	**0.37**	**0.41**	**0.40**	**0.31**
GLiNER-M	0.31	0.29	0.33	0.48	0.40	0.50	0.66	0.76	0.77	0.79	0.69	0.82	0.75	0.58
GLiNER-L	0.68	0.67	0.77	1.04	0.91	1.01	1.37	1.51	1.54	1.49	1.38	1.52	1.43	1.18
GNER	17.13	24.28	39.75	50.88	44.76	20.60	17.02	37.17	49.11	63.50	45.22	38.69	26.50	36.51
Qwen2.5-1.5B‡	12.7	15.1	17.5	18.4	18.0	7.9	4.6	9.5	7.0	11.1	7.3	10.2	9.5	11.4
OWNER§	6.6	9.0	12.7	18.3	12.6	6.5	8.6	16.3	13.4	17.1	22.0†	23.2	19.1	14.3
OPENER-ZS (base)	0.95	1.06	1.37	1.97	1.69	1.52	2.07	2.28	2.15	2.30	2.10	2.33	2.26	1.85
OPENER-ZS	1.14	1.23	1.58	2.08	1.82	1.41	1.82	2.23	1.62	1.87	1.63	1.90	1.79	1.70
Supervised (target labels)														
OPENER-Sup	1.10	1.26	0.87	1.76	1.41	1.31	1.75	1.87	1.96	1.97	1.73	1.91	1.78	1.59

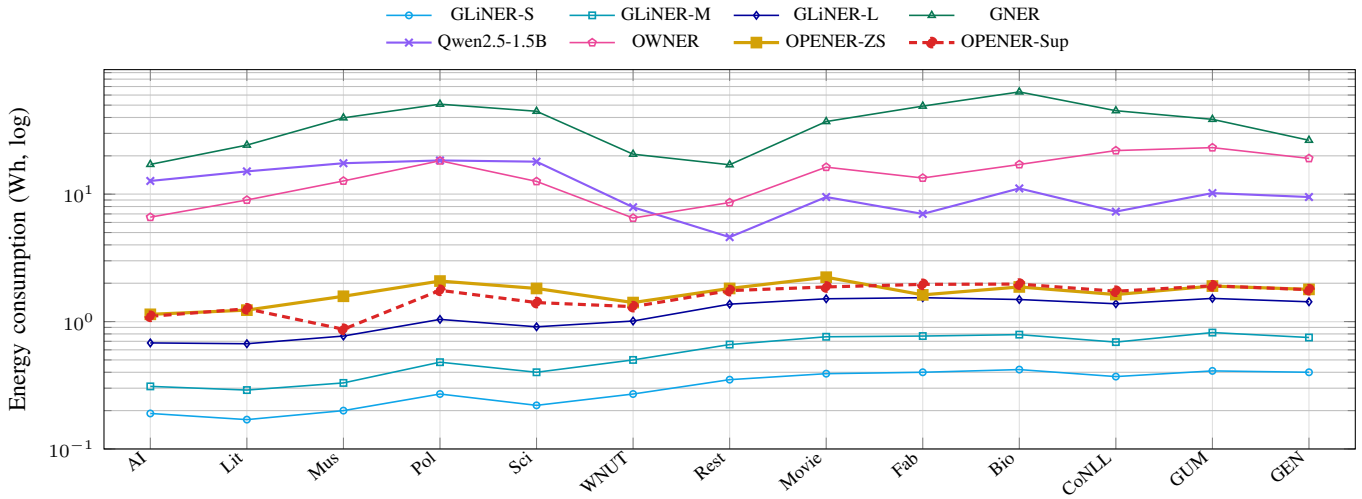

Fig. 4. Per-run energy consumption (Wh, *log* scale) per dataset (Table VI). Same systems and styling as Figure 2. The OPENER variants sit an order of magnitude below OWNER and the generative GNER/Qwen baselines.

TABLE VII

QUALITY VS EFFICIENCY, 13-SET MEANS. AMI (×100, ↑); P50 LATENCY (MS), ENERGY (WH), CARBON (gCO_2eq) PER RUN (↓); PARAMETERS (END-TO-END OPENER = GLINER-L DETECTOR 330M + EMBEDDER 137M = 467M, OR 137M ON GOLD; OWNER ≈294M; GLINER SIZES FROM PROJECT NOTES). [§] OWNER'S LATENCY IS AN AMORTISED WALL-CLOCK MEAN AND ITS ONE-TIME 88-MIN TRAINING IS NOT CHARGED PER RUN (TABLE V). [‡] QWEN2.5-1.5B-INSTRUCT (4-BIT) IS ON A 150-SENTENCE SUBSAMPLE PER SET.

Model	Params	AMI	p50 (ms)	Energy (Wh)	CO_2 (g)
Zero-shot (no target labels)					
GLiNER-S	50M	36.5	21	0.31	0.02
GLiNER-M	200M	37.1	40	0.58	0.03
GLiNER-L	330M	38.9	143	1.18	0.07
GNER	220M	37.9	4226	36.51	2.05
Qwen2.5-1.5B (4-bit)[‡]	1.5B	26.1	5051	11.4	0.60
OWNER[§]	294M	34.3	616	14.3	0.74
OPENER-ZS (base)	467M	34.2	137	1.85	0.10
OPENER-ZS	467M	39.4	182	1.70	0.10
Supervised (target labels)					
OPENER-Sup	467M	40.2	143	1.59	0.09

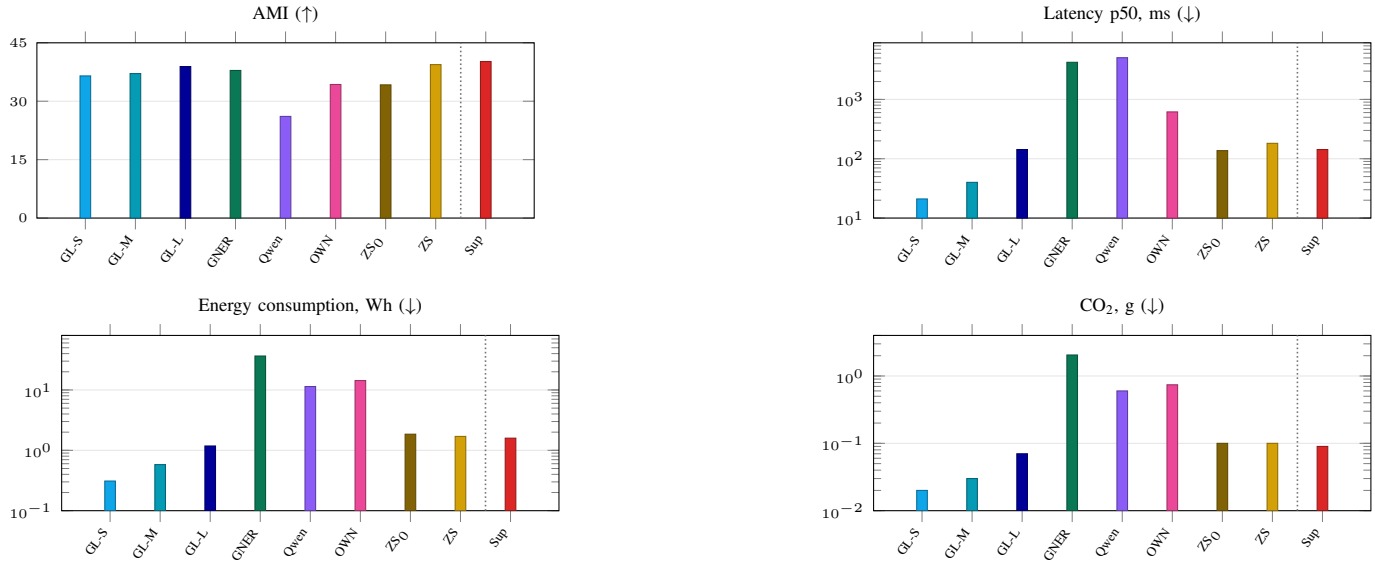

Fig. 5. Per-model means on the four axes (13-set averages, Table VII); the light dotted line separates the *zero-shot* systems (left) from the *supervised* OPENER-Sup (right). Latency, energy and CO_2 use a log scale. Labels: GL-S/M/L = GLiNER sizes, OWN = OWNER, ZS_0 = OPENER-ZS (base), ZS = OPENER-ZS, Sup = OPENER-Sup.

what caps end-to-end quality is the open-world detector's recall on cryptic, domain-specific surface forms, which confirms detection, not typing, as the dominant bottleneck.

c) What remains hard: The residual typing errors are dominated by confusions between fine-grained, lexically close types *within* a domain rather than across domains. The most frequent type confusions are DNA vs. protein (BioNLP), the FabNER process labels CONPRI vs. MATE/PRO/MANP, and the Movie attributes TITLE vs. GENRE/RATING: precisely the opaque schemas of Table I where even the surrounding context barely disambiguates the type.

d) Contrastive learning in action: Figure 6 visualises the effect of the two fine-tuning stages on a *held-out* domain (WNUT-17), never seen during contrastive training. Frozen off-the-shelf, the Nomic embedding intermixes entity types; the contrastive step already pulls same-type mentions together, and the hard-negative mining pass sharpens the boundaries between the types it most often confuses, yielding compact, well-separated clusters. The type geometry learned on CoNLL thus transfers to an unseen domain, which is what makes the cheap label-name and linear typing heads effective.

H. Ablations

We justify each architectural choice with a one-factor-at-a-time search. Starting from the default pipeline, we vary a single component while holding the others fixed, and keep the setting with the highest mean AMI. Among embedder variants, an

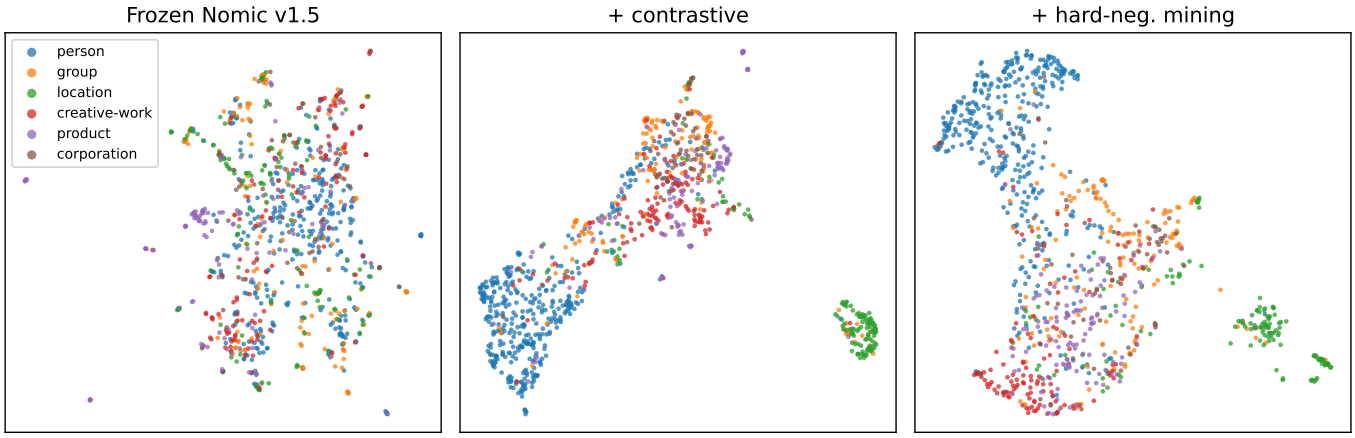

Fig. 6. UMAP of WNUT-17 (held-out) entity embeddings, coloured by gold type, across the three embedder stages: frozen Nomic v1.5, + contrastive fine-tuning, + hard-negative mining. The contrastive stages progressively separate the types into compact clusters on a domain unseen during training.

error-driven *hard-negative mining* pass, fine-tuning the contrastive embedder on the entity pairs it most often confuses (e.g. *writer/person*, *album/band*, *country/location*), mixed with correctly-typed mentions for diversity, raises typing-on-gold AMI from 54.0 to 60.1; scaling the mined triplets ($3000 \rightarrow 8000$) and epochs ($2 \rightarrow 3$) lifts it further to 62.5 ($+8.5$ over the contrastive embedder, better on 12 of 13 sets), our selected configuration. Table VIII reports this search; the protocol comparison between typing-on-gold and end-to-end is discussed alongside the main results. The remaining axes confirm the rest of the pipeline: among frozen off-the-shelf base encoders, Nomic v1.5 is the most discriminative for entity typing (25.8 AMI, ahead of e5-base 25.3, mxbai 24.4, MPNet 24.1 and BGE 23.6) and is selected (the heaviest candidate, mxbai-large, is moreover $\sim 8\times$ slower per sentence for no quality gain); even so, frozen embeddings are only weakly discriminative, so the contrastive step is what makes the space usable ($+28.2$); the detector—which only proposes spans, re-typed by the OPENER head—trades quality for cost (GLiNER-S $\rightarrow$ L: $37.7 \rightarrow 40.2$ AMI at $51 \rightarrow 143$ ms end-to-end), and OPENER's typing on GLiNER-L's spans (40.2) even beats GLiNER-L's own typing (38.9, Table IV); a closed-set NER detector (BERT- or RoBERTa-NER, CoNLL-trained) used in its place collapses to ~ 0 recall outside PER/ORG/LOC/MISC (MIT, FabNER, BioNLP), dragging macro-F_1 to 17.9/24.6 (vs 30.5) and confirming that open-world detection is required; and for typing a balanced linear SVM edges out a per-class GMM and logistic regression (62.5 vs 56.1 vs 55.6). Replacing the supervised typing head with a label-name nearest-centroid removes target supervision entirely, at a cost of 18.1 AMI ($62.5 \rightarrow 44.4$); this base variant, OPENER-ZS (base), still surpasses the zero-shot OWNER baseline (43.0, Table IX) while fitting in 293 ms instead of 3.5 s, making it the frugal choice when no target labels are available. A label-free *transductive* refinement of the prototypes — spherical k-means seeded by the label names and run once on the unlabelled test mentions — recovers 7.0 AMI on gold mentions ($44.4 \rightarrow 51.4$, Table VIII) and 2.6 end-to-end ($34.2 \rightarrow 36.8$, above OWNER's 34.3), widening OPENER-ZS's margin over OWNER while still using no target labels; this is a transductive rather than inductive setting, as it exploits the unlabelled test distribution. We further probed whether the zero-shot head could be improved by aligning entity mentions and label *names* in a shared space, trained on the large label vocabulary of Pile-NER [3] ($\approx$4.6k types) with an entity$\leftrightarrow$label-name contrastive objective (Table X). This produces a *strictly* domain-agnostic embedder (unlike the selected hard-mining embedder, it never sees any target-side data), yet it does not improve mean AMI (42.2 vs 44.4): the Pile-NER vocabulary lifts macro-F_1 ($+3.8$) and helps encyclopedic domains (CrossNER-Politics $+14.2$, -Science $+8.6$) but hurts specialised schemas (MIT-Restaurant -15.9, FabNER -11.8, BioNLP -11.8). A second label-name hard-mining pass degrades it further (39.7 AMI): Pile-NER's GPT-generated types include many near-synonyms (e.g. *person/per*, *location/loc*), so mining "confusions" between them manufactures negatives that pull apart labels that should stay close. We therefore retain the target-tuned hard-mining embedder for OPENER-ZS and report the Pile-NER variants as a negative ablation. A separate efficiency axis emerges in the *training* cost (Table VIII, Fit): per-sentence inference is governed by the detector alone (truncating the Matryoshka dimension or swapping the classifier leaves it unchanged), whereas fitting the typing head is $7\times$ cheaper at dimension 64 than at 768 (for only a 4.6-point AMI drop, $62.5 \rightarrow 57.9$) and $4.5\times$ cheaper for balanced logistic regression than for the linear SVM, a speed/quality knob orthogonal to inference. To isolate the entity-typing component itself, Table IX compares OPENER against OWNER on gold mentions, where detection errors are removed.

TABLE VIII

OPENER ARCHITECTURE SEARCH: ONE COMPONENT IS VARIED AT A TIME AROUND THE DEFAULT PIPELINE (**BOLD**). AMI AND MACRO-F_1 ($\times 100$, $\uparrow$); FIT (MS) IS THE TYPING-HEAD TRAINING TIME; PER-ROW LATENCY, ENERGY AND CARBON ($\downarrow$). ON THE *mention-detector* AXIS, EACH ROW IS OPENER'S *end-to-end* AMI WITH THAT DETECTOR—IT ONLY PROPOSES SPANS, RE-TYPED BY THE OPENER HEAD (UNLIKE THE STANDALONE SYSTEMS OF TABLE IV). THE * EMBEDDER, ‡ TYPING AND § DIMENSION AXES ARE TYPING-ON-GOLD OVER THE 13 SETS (§ TRUNCATES THE MATRYOSHKA VECTOR AFTER ENCODING); ¶ MARKS THE ZERO-SHOT NEAREST-CENTROID HEAD (NO TARGET LABELS; ITS FIT IS THE ONE-OFF PROTOTYPE-EMBEDDING COST). LATENCY/ENERGY IS THE DEPLOYED END-TO-END COST ON THE MD AXIS AND THE CONSTANT EMBEDDER-PLUS-TYPING COST ELSEWHERE; PER-AXIS FINDINGS ARE ANALYSED IN SUBSECTION IV-H.

Configuration	AMI	F_1	Fit (ms)	p50 (ms)	En. (Wh)	CO_2 (g)
Mention detector (each row: that detector $\rightarrow$ OPENER embedder + typing head)						
GLiNER-S	37.7	28.8	0.0	51	0.73	0.04
GLiNER-M	38.0	28.7	0.0	70	1.01	0.06
GLiNER-L (open-world)	**40.2**	**30.5**	0.0	143	1.59	0.09
BERT-NER (closed-set) [7]	33.9	17.9	0.0	68	0.58	0.03
RoBERTa-NER (closed-set) [60]	39.4	24.6	0.0	54	0.58	0.04
*Entity embedder**						
frozen bge-base-en-v1.5 [30]	23.6	41.9	0	43	0.63	0.04
frozen all-mpnet-base-v2 [31]	24.1	42.6	0	52	0.85	0.05
frozen mxbai-embed-large-v1 [32]	24.4	42.5	0	363	3.86	0.25
frozen e5-base-v2 [29]	25.3	43.4	0	39	0.59	0.04
frozen Nomic v1.5 (selected base)	25.8	43.9	0	42	0.63	0.04
+ contrastive (Triplet, CoNLL, 3 ep.)	54.0	63.3	0	42	0.63	0.04
+ hard-negative mining (2 ep., 3k tr.)	60.1	68.8	0	42	0.63	0.04
+ hard-negative mining (3 ep., 8k tr.)	**62.5**	**71.8**	0	42	0.63	0.04
Matryoshka dimension§						
64	57.9	67.7	299	42	0.63	0.04
128	59.9	69.7	562	42	0.63	0.04
256	61.2	70.6	926	42	0.63	0.04
512	62.0	71.2	1556	42	0.63	0.04
768	**62.5**	**71.8**	2176	42	0.63	0.04
Entity typing‡						
GMM (per-class)	56.1	65.3	527	42	0.63	0.04
LogReg	55.6	53.5	546	42	0.63	0.04
LogReg (balanced)	57.8	67.5	485	42	0.63	0.04
LinearSVC	61.7	66.5	2041	42	0.63	0.04
LinearSVC (balanced)	**62.5**	**71.8**	2217	42	0.63	0.04
Nearest centroid (zero-shot)¶	44.4	46.8	293	42	0.63	0.04
+ transductive refine¶	51.4	48.5	293	42	0.63	0.04

TABLE IX

ENTITY TYPING ON GOLD MENTIONS (DETECTION BYPASSED): OPENER VS OWNER (AMI $\times 100$, $\uparrow$). ABBREVIATIONS FOLLOW TABLE I; BOLD MARKS THE BEST VALUE PER COLUMN; SYSTEMS ARE GROUPED BY SUPERVISION. HERE OPENER-ZS IS THE TRANSDUCTIVELY-REFINED PROTOTYPE HEAD (THE DETECTOR FUSION OF TABLE IV IS END-TO-END-ONLY, INACTIVE WITHOUT A DETECTOR); OPENER-ZS (BASE) IS THE UN-REFINED NEAREST CENTROID; OPENER-SUP IS THE SUPERVISED UPPER BOUND. † CONLL-2003 IS IN-DOMAIN FOR OWNER (TRAINING SOURCE) AND OPENER (CONTRASTIVE SOURCE) AND IS NOT BOLDED.

Model	AI	Lit	Mus	Pol	Sci	WNUT	Rest	Movie	Fab	Bio	CoNLL	GUM	GEN	Avg
Zero-shot (no target labels)														
OWNER	55.7	54.3	59.8	63.2	63.1	30.5	29.3	31.8	31.9	29.2	52.5†	30.0	27.2	43.0
OPENER-ZS (base)	41.4	61.4	55.2	50.7	58.0	33.0	42.6	18.4	35.6	54.1	65.9†	31.1	30.2	44.4
OPENER-ZS	46.4	65.0	68.6	66.5	57.3	31.8	52.1	45.8	37.5	49.5	71.0†	38.6	37.5	51.4
Supervised (target labels)														
OPENER-Sup	**57.6**	**67.0**	**71.8**	**74.8**	**68.9**	**38.6**	**63.0**	**66.9**	**59.2**	**71.2**	71.5†	**50.3**	**51.5**	**62.5**

TABLE X

ZERO-SHOT TYPING: EFFECT OF THE CONTRASTIVE EMBEDDER ON OPENER-ZS (TYPING-ON-GOLD, 13-SET MEANS; THE NEAREST LABEL-NAME CENTROID HEAD IS FIXED, ONLY THE EMBEDDER CHANGES). AMI, MACRO-F_1, ACCURACY ($\times 100$, $\uparrow$); BEST PER COLUMN IN BOLD. THE SELECTED HARD-NEGATIVE-MINING EMBEDDER (TARGET-TUNED) IS COMPARED WITH PILE-NER [3] VARIANTS THAT ARE STRICTLY DOMAIN-AGNOSTIC (NEVER SEE TARGET DATA); WE REPORT PILE-NER AS A NEGATIVE ABLATION (SUBSECTION IV-H).

Embedder	Training data	AMI	F_1	Acc
hard-neg. mining (selected)	CoNLL + targets' train	**44.4**	46.8	53.6
Pile-NER (entity↔label)	Pile-NER only	42.2	**50.6**	**53.9**
+ label-name hard mining	Pile-NER only	39.7	41.4	44.7

V. CONCLUSION

We presented OPENER, an open-world NER pipeline built from off-the-shelf components — a frozen detector, a pretrained Matryoshka embedder sharpened by a light contrastive fine-tuning with error-driven hard-negative mining, and a swappable typing head — with no encoder trained from scratch. On gold mentions the contrastive space is highly discriminative, and a balanced linear probe, OPENER-Sup, is the most accurate system in our benchmark, leading on all 13 sets and surpassing a zero-shot transfer of OWNER (62.5 vs 43.0 AMI); it also attains the best end-to-end AMI of the compared systems (40.2), at the cost of target labels. Removing that supervision, our zero-shot head OPENER-ZS — label-name prototypes refined transductively and fused with the detector's own predictions — is the best of the compared zero-shot systems end-to-end (39.4), ahead of GLiNER-L, GNER and OWNER, at one to two orders of magnitude lower energy than the trained-encoder and generative baselines.

Two findings carry beyond the specific system. First, an open-world typing head need not train its own encoder: a general-purpose embedding, once sharpened by a light contrastive step, is already discriminative enough to type entities, and the cost that does matter is fitting the cheap typing head, not the embedder. Second, the three-axis evaluation locates the open-world bottleneck in *detection* rather than typing: typing on gold mentions reaches 62.5 AMI but end-to-end scores drop to 40.2 as detector recall falls on specialised domains, so improving the detector—not the typing head—is the clearest axis for future gains. The evaluation also exposes efficiency gaps that an accuracy-only comparison would hide.

The approach has clear limits. End-to-end quality is capped by detector recall on specialised domains, the shared bottleneck, not by the typing head. Supervised typing needs target-domain labels, and while OPENER-ZS removes them at typing time it depends on the quality of the label names and, in its transductive form, on access to the unlabelled test mentions; the inductive zero-shot head is correspondingly weaker. Finally, our experiments run on deliberately modest hardware: a single thermally-throttling consumer GPU with 6 GB of VRAM. This is the very setting that makes OPENER deployable, but it also bounded our experimental budget: despite an already broad benchmark (13 datasets, three axes and extensive ablations), we report a single random seed and cap each test split at 1000 sentences, so per-run figures carry run-to-run variance and are read as averages rather than precise points. A larger compute budget would extend the same protocol to multiple random seeds with variance and significance tests, the full uncapped test splits, and a controlled-temperature re-measurement of latency and energy.

Future work includes improving mention detection on specialised domains to lift the end-to-end ceiling, an inductive zero-shot head that does not rely on the test distribution, a visualisation of the contrastive type geometry (e.g. UMAP or t-SNE), and extending the decoupled detect-then-type design beyond NER to entity linking and relation extraction.

ACKNOWLEDGMENTS

In accordance with the symposium policy on the use of generative AI, we disclose that a general-purpose large-language-model coding assistant was used during this work. Its use was limited to engineering and editorial support: scaffolding and debugging experiment scripts, drafting and copy-editing the manuscript, and assisting with exploratory data analysis. The research questions, the experimental design, the choice of baselines and datasets, the execution of all experiments, and the interpretation of the results are the authors' own. All reported numbers were produced by the released code and verified by the authors, who take full responsibility for the scientific content and the final wording of the paper.

REFERENCES

[1] N. T. H. Nguyen, M. Miwa, and S. Ananiadou, "Span-based named entity recognition by generating and compressing information," in *Proceedings of the 17th Conference of the European Chapter of the Association for Computational Linguistics (EACL)*, 2023, pp. 1984–1996.

[2] P.-Y. Genest, P.-E. Portier, E. Egyed-Zsigmond, and M. Lovisetto, "OWNER: Toward unsupervised open-world named entity recognition," *IEEE Access*, vol. 13, pp. 50 077–50 105, 2025.

[3] W. Zhou, S. Zhang, Y. Gu, M. Chen, and H. Poon, "UniversalNER: Targeted distillation from large language models for open named entity recognition," in *The Twelfth International Conference on Learning Representations (ICLR)*, 2024.

[4] O. Sainz, I. García-Ferrero, R. Agerri, O. Lopez de Lacalle, G. Rigau, and E. Agirre, "GoLLIE: Annotation guidelines improve zero-shot information-extraction," in *The Twelfth International Conference on Learning Representations (ICLR)*, 2024.

[5] G. Lample, M. Ballesteros, S. Subramanian, K. Kawakami, and C. Dyer, "Neural architectures for named entity recognition," in *Proceedings of the 2016 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (NAACL-HLT)*, 2016, pp. 260–270.

[6] X. Ma and E. Hovy, "End-to-end sequence labeling via bi-directional LSTM-CNNs-CRF," in *Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics (ACL)*, 2016, pp. 1064–1074.

[7] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of deep bidirectional transformers for language understanding," in *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics (NAACL)*, 2019.

[8] J. Yu, B. Bohnet, and M. Poesio, "Named entity recognition as dependency parsing," in *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics (ACL)*, 2020, pp. 6470–6476.

[9] X. Li, J. Feng, Y. Meng, Q. Han, F. Wu, and J. Li, "A unified MRC framework for named entity recognition," in *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics (ACL)*, 2020, pp. 5849–5859.

[10] E. F. Tjong Kim Sang and F. De Meulder, "Introduction to the CoNLL-2003 shared task: Language-independent named entity recognition," in *Proceedings of the Seventh Conference on Natural Language Learning (CoNLL)*, 2003.

[11] J. Li, A. Sun, J. Han, and C. Li, "A survey on deep learning for named entity recognition," *IEEE Transactions on Knowledge and Data Engineering*, vol. 34, no. 1, pp. 50–70, 2022.

[12] U. Zaratiana, N. Tomeh, P. Holat, and T. Charnois, "GLiNER: Generalist model for named entity recognition using bidirectional transformer," in *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (NAACL-HLT), Volume 1: Long Papers*. Association for Computational Linguistics, 2024, pp. 5364–5376.

[13] P. He, J. Gao, and W. Chen, "DeBERTaV3: Improving DeBERTa using ELECTRA-style pre-training with gradient-disentangled embedding sharing," in *The Eleventh International Conference on Learning Representations (ICLR)*, 2023.

[14] Y. Ding, J. Li, P. Wang, Z. Tang, B. Yan, and M. Zhang, "Rethinking negative instances for generative named entity recognition," in *Findings of the Association for Computational Linguistics: ACL 2024*, 2024, pp. 3461–3475.

[15] C. Raffel, N. Shazeer, A. Roberts, K. Lee, S. Narang, M. Matena, Y. Zhou, W. Li, and P. J. Liu, "Exploring the limits of transfer learning with a unified text-to-text transformer," *Journal of Machine Learning Research*, vol. 21, pp. 1–67, 2020.

[16] R. Aly, A. Vlachos, and R. McDonald, "Leveraging type descriptions for zero-shot named entity recognition and classification," in *Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics and the 11th International Joint Conference on Natural Language Processing (ACL-IJCNLP)*, 2021, pp. 1516–1528.

[17] N. Ding, G. Xu, Y. Chen, X. Wang, X. Han, P. Xie, H.-T. Zheng, and Z. Liu, "Few-NERD: A few-shot named entity recognition dataset," in *Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics and the 11th International Joint Conference on Natural Language Processing (ACL-IJCNLP)*, 2021, pp. 3198–3213.

[18] X. Wei, X. Cui, N. Cheng, X. Wang, X. Zhang, S. Huang, P. Xie, J. Xu, Y. Chen, M. Zhang, Y. Jiang, and W. Han, "ChatIE: Zero-shot information extraction via chatting with ChatGPT," *arXiv preprint arXiv:2302.10205*, 2023.

[19] R. Cai, J. Lu, Z. Chen, B. Xu, and Z. Hao, "Handling missing entities in zero-shot named entity recognition: Integrated recall and retrieval augmentation," in *Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics: Human Language Technologies (NAACL)*, 2025, pp. 10 790–10 802.

[20] H. Touvron, T. Lavril, G. Izacard, X. Martinet, M.-A. Lachaux, T. Lacroix, B. Rozière, N. Goyal, E. Hambro, F. Azhar, A. Rodriguez, A. Joulin, E. Grave, and G. Lample, "LLaMA: Open and efficient foundation language models," *arXiv preprint arXiv:2302.13971*, 2023.

[21] Y. Tang, R. Hasan, and T. Runkler, "FsPONER: Few-shot prompt optimization for named entity recognition in domain-specific scenarios," in *Proceedings of the 27th European Conference on Artificial Intelligence (ECAI)*, 2024, arXiv:2407.08035.

[22] S. Zhang, B. Cao, and J. Fan, "KCL: Few-shot named entity recognition with knowledge graph and contrastive learning," in *Proceedings of the 2024 Joint International Conference on Computational Linguistics, Language Resources and Evaluation (LREC-COLING)*. ELRA Language Resource Association, 2024, pp. 9681–9692.

[23] S. S. S. Das, A. Katiyar, R. J. Passonneau, and R. Zhang, "CONTaiNER: Few-shot named entity recognition via contrastive learning," in *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (ACL)*, 2022.

[24] Qwen Team, "Qwen2.5 technical report," *arXiv preprint arXiv:2412.15115*, 2025.

[25] Z. Nussbaum, J. X. Morris, B. Duderstadt, and A. Mulyar, "Nomic embed: Training a reproducible long context text embedder," *Transactions on Machine Learning Research (TMLR)*, 2025, published in TMLR 02/2025. OpenReview: https://openreview.net/forum?id=IPmzyQSiQE.

[26] A. Kusupati, G. Bhatt, A. Rege, M. Wallingford, A. Sinha, V. Ramanujan, W. Howard-Snyder, K. Chen, S. Kakade, P. Jain, and A. Farhadi, "Matryoshka representation learning," in *Advances in Neural Information Processing Systems 35 (NeurIPS 2022)*, 2022.

[27] N. Reimers and I. Gurevych, "Sentence-BERT: Sentence embeddings using siamese BERT-networks," in *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2019.

[28] F. Schroff, D. Kalenichenko, and J. Philbin, "FaceNet: A unified embedding for face recognition and clustering," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2015.

[29] L. Wang, N. Yang, X. Huang, B. Jiao, L. Yang, D. Jiang, R. Majumder, and F. Wei, "Text embeddings by weakly-supervised contrastive pre-training," *arXiv preprint arXiv:2212.03533*, 2022.

[30] S. Xiao, Z. Liu, P. Zhang, N. Muennighoff, D. Lian, and J.-Y. Nie, "C-Pack: Packed resources for general chinese embeddings," in *Proceedings of the 47th International ACM SIGIR Conference on Research and Development in Information Retrieval (SIGIR)*, 2024.

[31] K. Song, X. Tan, T. Qin, J. Lu, and T.-Y. Liu, "MPNet: Masked and permuted pre-training for language understanding," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.

[32] X. Li and J. Li, "AoE: Angle-optimized embeddings for semantic textual similarity," in *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (ACL)*, 2024.

[33] G. Wang, C. Li, W. Wang, Y. Zhang, D. Shen, X. Zhang, R. Henao, and L. Carin, "Joint embedding of words and labels for text classification," in *Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics (ACL)*, 2018, pp. 2321–2331.

[34] Y. Zhang, W. Chen, and L. Ma, "Contrastive learning with Gaussian embeddings and self-attention for few-shot named entity recognition," *Applied Sciences*, vol. 15, no. 21, p. 11819, 2025.

[35] X. Hu, Y. Jiang, A. Liu, Z. Huang, P. Xie, F. Huang, L. Wen, and P. S. Yu, "Entity-to-text based data augmentation for various named entity recognition tasks," in *Findings of the Association for Computational Linguistics: ACL 2023*, 2023, pp. 9072–9087.

[36] Y. Abbahaddou, F. D. Malliaros, J. F. Lutzeyer, A. M. Aboussalah, and M. Vazirgiannis, "Gaussian mixture models based augmentation enhances GNN generalization," 2024, arXiv:2411.08638 [cs.LG], preprint.

[37] D. A. Reynolds, "Gaussian mixture models," in *Encyclopedia of Biometrics*, S. Z. Li and A. K. Jain, Eds. Springer US, 2009, pp. 659–663.

[38] A. P. Dempster, N. M. Laird, and D. B. Rubin, "Maximum likelihood from incomplete data via the EM algorithm," *Journal of the Royal Statistical Society: Series B (Methodological)*, vol. 39, no. 1, pp. 1–22, 1977.

[39] E. Strubell, A. Ganesh, and A. McCallum, "Energy and policy considerations for deep learning in NLP," in *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics (ACL)*, 2019.

[40] R. Schwartz, J. Dodge, N. A. Smith, and O. Etzioni, "Green AI," *Communications of the ACM*, vol. 63, no. 12, pp. 54–63, 2020.

[41] D. Patterson, J. Gonzalez, U. Hölzle, Q. Le, C. Liang, L.-M. Munguia, D. Rothchild, D. So, M. Texier, and J. Dean, "The carbon footprint of machine learning training will plateau, then shrink," *Computer*, vol. 55, no. 7, pp. 18–28, 2022.

[42] P. Henderson, J. Hu, J. Romoff, E. Brunskill, D. Jurafsky, and J. Pineau, "Towards the systematic reporting of the energy and carbon footprints of machine learning," *Journal of Machine Learning Research*, vol. 21, pp. 1–44, 2020.

[43] C.-J. Wu, R. Raghavendra, U. Gupta, B. Acun, N. Ardalani, K. Maeng, G. Chang, F. Aga, J. Huang, C. Bai *et al.*, "Sustainable AI: Environmental implications, challenges and opportunities," in *Proceedings of Machine Learning and Systems (MLSys)*, 2022.

[44] A. S. Luccioni, Y. Jernite, and E. Strubell, "Power hungry processing: Watts driving the cost of AI deployment?" in *Proceedings of the 2024 ACM Conference on Fairness, Accountability, and Transparency (FAccT)*, 2024.

[45] A. Lacoste, A. Luccioni, V. Schmidt, and T. Dandres, "Quantifying the carbon emissions of machine learning," *arXiv preprint arXiv:1910.09700*, 2019.

[46] N. X. Vinh, J. Epps, and J. Bailey, "Information theoretic measures for clusterings comparison: Variants, properties, normalization and correction for chance," *Journal of Machine Learning Research*, vol. 11, pp. 2837–2854, 2010.

[47] R.-E. Fan, K.-W. Chang, C.-J. Hsieh, X.-R. Wang, and C.-J. Lin, "LIBLINEAR: A library for large linear classification," *Journal of Machine Learning Research*, vol. 9, pp. 1871–1874, 2008.

[48] Z. Liu, Y. Xu, T. Yu, W. Dai, Z. Ji, S. Cahyawijaya, A. Madotto, and P. Fung, "CrossNER: Evaluating cross-domain named entity recognition," in *Proceedings of the AAAI Conference on Artificial Intelligence*, 2021.

[49] L. Derczynski, E. Nichols, M. van Erp, and N. Limsopatham, "Results of the WNUT2017 shared task on novel and emerging entity recognition," in *Proceedings of the 3rd Workshop on Noisy User-generated Text (W-NUT)*, 2017, pp. 140–147.

[50] J. Liu, P. Pasupat, S. Cyphers, and J. Glass, "Asgard: A portable architecture for multilingual dialogue systems," in *Proceedings of the IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, 2013, pp. 8386–8390.

[51] A. Kumar and B. Starly, "FabNER: Information extraction from manufacturing process science domain literature using named entity recognition," *Journal of Intelligent Manufacturing*, vol. 33, no. 8, pp. 2393–2407, 2022.

[52] J.-D. Kim, T. Ohta, Y. Tsuruoka, Y. Tateisi, and N. Collier, "Introduction to the bio-entity recognition task at JNLPBA," in *Proceedings of the International Joint Workshop on Natural Language Processing in Biomedicine and its Applications (NLPBA/BioNLP)*, 2004, pp. 70–75.

[53] A. Zeldes, "The GUM corpus: Creating multilayer resources in the classroom," *Language Resources and Evaluation*, vol. 51, no. 3, pp. 581–612, 2017.

[54] T. Aoyama, S. Behzad, L. Gessler, L. Levine, J. Lin, Y. J. Liu, S. Peng, Y. Zhu, and A. Zeldes, "GENTLE: A genre-diverse multilayer challenge set for english NLP and linguistic evaluation," in *Proceedings of the 17th Linguistic Annotation Workshop (LAW-XVII)*, 2023.

[55] J.-D. Kim, T. Ohta, Y. Tateisi, and J. Tsujii, "GENIA corpus—a semantically annotated corpus for bio-textmining," *Bioinformatics*, vol. 19, no. Suppl. 1, pp. i180–i182, 2003.

[56] T. Dettmers, A. Pagnoni, A. Holtzman, and L. Zettlemoyer, "QLoRA: Efficient finetuning of quantized LLMs," *arXiv preprint arXiv:2305.14314*, 2023.

[57] A. Paszke, S. Gross, F. Massa, A. Lerer, J. Bradbury, G. Chanan, T. Killeen, Z. Lin, N. Gimelshein, L. Antiga, A. Desmaison, A. Köpf, E. Yang, Z. DeVito, M. Raison, A. Tejani, S. Chilamkurthy, B. Steiner, L. Fang, J. Bai, and S. Chintala, "PyTorch: An imperative style, high-performance deep learning library," in *Advances in Neural Information Processing Systems 32 (NeurIPS)*, 2019.

[58] T. Wolf, L. Debut, V. Sanh, J. Chaumond, C. Delangue, A. Moi, P. Cistac, T. Rault, R. Louf, M. Funtowicz, J. Davison, S. Shleifer, P. von Platen, C. Ma, Y. Jernite, J. Plu, C. Xu, T. Le Scao, S. Gugger, M. Drame, Q. Lhoest, and A. M. Rush, "Transformers: State-of-the-art natural language processing," in *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing: System Demonstrations (EMNLP)*, 2020.

[59] F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, M. Blondel, P. Prettenhofer, R. Weiss, V. Dubourg, J. Vanderplas, A. Passos, D. Cournapeau, M. Brucher, M. Perrot, and É. Duchesnay, "Scikit-learn: Machine learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.

[60] Y. Liu, M. Ott, N. Goyal, J. Du, M. Joshi, D. Chen, O. Levy, M. Lewis, L. Zettlemoyer, and V. Stoyanov, "RoBERTa: A robustly optimized BERT pretraining approach," *arXiv preprint arXiv:1907.11692*, 2019.